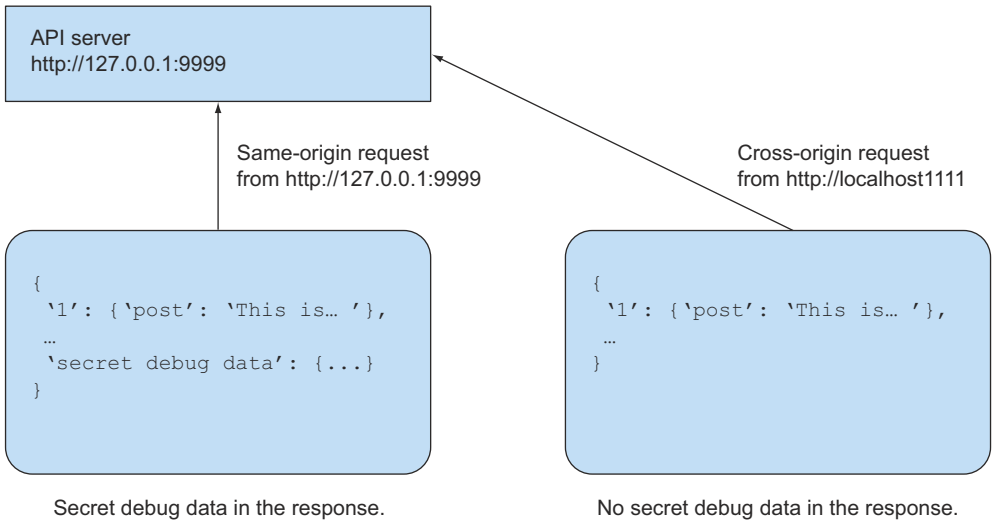



**Figure 6.8 Different data is sent to same-origin and cross-origin requests**

But if same-origin requests behave differently from cross-origin requests, the distinction between the two becomes relevant. Suppose you have an API that accepts requests from your own web page (a same-origin request) as well as requests from other clients (a cross-origin request). The request from your own web page receives additional debug information, which you don't want to share with cross-origin clients, as shown in figure 6.8. In this case, you want to differentiate between same-origin and cross-origin requests; otherwise cross-origin requests could receive data they aren't supposed to see.

In these cases, you can compare the Origin header against the Host header to see if they match. The Host header is an HTTP request header that contains the host and port of the server receiving the request. Because the Origin header contains the scheme, host, and port of the client sending the request, if the Origin and the Host headers match, the request is a same-origin request.

The following code snippet shows a function for detecting whether the request is a same-origin request. The `isSameOrigin` function returns `true` if the request is a same-origin request, and `false` otherwise.

```
var isSameOrigin = function(req) {
  var host = req.protocol + '://' + req.headers['host'];
  var origin = req.headers['origin'];
  return host === origin || !origin;
};
```

The `isSameOrigin` function extracts the client origin from the Origin header and compares it to the server's origin, which is derived from the Host header. Note that the Host header doesn't include any scheme information, so it's up to you to decide which scheme to use. The preceding code snippet uses the `req.protocol` property to